

Habit Tracker API

Bulut Mimarilerinde Test Muhendisligi — Final Rapor

Ogrenci	Mert Baytas
Ders	MTH2526-B25 — Bulut Mimarilerinde Test Muhendisligi
Egitmen	Busra Ayaksiz
Universite	Marmara Universitesi — Bilgisayar Muhendisligi
Konu	Habit Tracker API (Gunluk aliskanlik tracking, streak hesapla)
Teslim	4 Haziran 2026
Repo	github.com/baytasmert/habit-tracker-api

1. Giris

Bu proje, Marmara Universitesi Bilgisayar Muhendisligi bolumunun "Bulut Mimarilerinde Test Muhendisligi" dersi kapsaminda gelistirilmis bireysel bir donem projesidir. Proje konusu olarak **Habit Tracker API** (konu #3) secilmistir: gunluk aliskanlik takibi, streak hesaplama ve kullanıcı istatistikleri sunan bir REST API.

Projenin amaci, "karmasik bir uygulama yazmak" degil, "basit bir uygulama icin endustri standardinda test ve dagitiim altyapisi kurmaktir." Bu dogrultuda FastAPI, PostgreSQL, LocalStack S3, Docker, Kubernetes, GitHub Actions, Prometheus/Grafana ve k6 gibi araclar bir arada kullanilmistir.

2. Mimari

Asagidaki diyagram, sistemin tum bileşenlerini gostermektedir. (docs/architecture.png dosyasinda tam cozunurluklu versiyon mevcuttur.)



Bileşen	Teknoloji	Port	Acıklama
REST API	FastAPI 0.110	:8000	Habit CRUD, Auth, S3, OTEL
Frontend Proxy	NGINX	:8001	Static dosyalar, template proxy
Veritabanı	PostgreSQL 16	:5432	Kalici veri, SQLAlchemy ORM
Object Storage	LocalStack S3	:4566	Avatar/foto yikleme
Tracing	Jaeger all-in-one	:16686	OpenTelemetry OTLP gRPC
Metrics	Prometheus	:9090	API metrikleri toplama
Dashboard	Grafana	:3000	4 panel: latency, errors, RPS
CI/CD	GitHub Actions	-	lint->test->build->deploy->smoke
Container Orch.	Kubernetes/Kind	-	Deployment+Service+ConfigMap

3. Test Stratejisi

Test piramidi yaklasimi benimsenmistir: tabanda cok sayida birim test, ortada entegrasyon testleri ve tepede az sayida E2E test bulunmaktadır. Toplam 92 test calismakta olup hedef coverage degeri olan %70in ustundedir.

Katman	Arac	Adet	Kapsam
Unit Tests	pytest	~40	Model validasyon, auth, streak logic
Integration Tests	pytest + TestClient	~46	API endpoint CRUD, S3, Factory Boy
Testcontainers	testcontainers-python	4	Gercek PostgreSQL container ile DB akis
E2E Tests	Playwright	6	Kayit, giris, habit olustur/track/edit/hata
API Tests	Postman/Newman	5+	Health, register, login, habits, track
Perf Tests	k6	2	Smoke (5VU/30s) + Load (50VU/60s)

3.1 Test Verisi Uretimi

Factory Boy ve Faker kutuphaneleri kullanilarak gercekci test verisi uretilmistir. UserFactory, HabitFactory ve HabitLogFactory siniflari ile her test izole veri kullanmakta ve testler arasi bagimlilik ortadan kaldirilmistir.

```
tests/factories.py: UserFactory, HabitFactory, HabitLogFactory
```

3.2 Testcontainers

Gercek bir PostgreSQL 16 container ayaga kaldirarak 4 entegrasyon testi calistirilmistir. Bu testler, sema olusturma, kullanıcı kaydi, habit CRUD ve veri kaliciligi senaryolarini kapsamaktadır. Linux/CI ortaminda otomatik olarak calisir.

4. CI/CD Pipeline

Job	Amac	Suretec	Timeout
lint	flake8 kod kalitesi kontrolu	-	10 dk
test	pytest + coverage >= 70%	lint	45 dk
build	Docker image insa ve GHCR push	lint + test	20 dk
deploy-and-test	Kind K8s deploy + Newman + Playwright 5000	5000	25 dk
smoke-test	k6 smoke (5VU/30s) - GHCR imajini kullanil	deploy-and-test	15 dk
load-test	k6 load (50VU/60s) - GHCR imajini kullanil	smoke-test	15 dk

Onemli tasarim kararlari: (1) smoke-test ve load-test artik docker build yapmayip build job'undan gelen ayni GHCR imajini kullanmaktadir — tutarlilik saglanmistir. (2) deploy-and-test job'u Kind cluster uzerinde gercek K8s deployment testi yapar. (3) Tum job'lar fail-fast prensibine gore zincirle bagli olup onceki job basarisiz olursa sonraki calismaz.

5. Monitoring ve Gözlemlenebilirlik

5.1 Prometheus Metrikleri

Metrik	Tür	Açıklama
http_requests_total	Counter	Toplam HTTP istegi sayısı (endpoint, method, status bazında)
http_request_duration_seconds	Histogram	İstek süre dağılımı (p50, p95, p99)
http_requests_in_progress	Gauge	Su an işlenen istek sayısı

5.2 Grafana Dashboard (4 Panel)

Panel	Görsellik	Metrik
Request Rate (5m)	Time series	rate(http_requests_total[5m])
Total Requests by Endpoint	Bar chart	sum by(endpoint) http_requests_total
Request Latency (p95, p99)	Time series	histogram_quantile(0.95/0.99, ...)
Business Metrics	Stat panel	Habit/kullanıcı oluşturma oranları

5.3 OpenTelemetry Distributed Tracing (+5 Bonus)

FastAPI ve SQLAlchemy otomatik olarak enstrumante edilmiştir. Spanlar OTLP gRPC protokolü ile Jaeger'a iletilmektedir (grpc://jaeger:4317). Her HTTP istek, DB sorguları dahil tam trace zincirleriyle görüntülenmektedir.

6. Performans Raporu

k6 ile iki senaryo çalıştırılmıştır. Sonuçlar aşağıdaki gibidir:

Test	VU	Süre	p(95)	p(99)	Hata Oranı	Sonuç
Smoke Test	5	30s	< 500ms	< 1000ms	< 1%	PASS
Load Test	10->50	60s	285ms	301ms	0%	PASS

Yorum: p(95)=285ms değeri 500ms threshold'unun çok altındadır. Yüksek latency gözlemlenen tek işlem bcrypt password hashing olup bu bir güvenlik gereksinimidir, optimize edilmesi önerilmez. 50 eşit VU altında sistem stabil kalmış, hiç check hatası oluşmamıştır.

7. Sonuc ve Ogrendiklerim

7.1 Projenin Sayisal Ozeti

Olcut	Deger
Toplam Test Sayisi	92 test
Test Coverage	>= 70% (hedef karsilandi)
E2E Senaryolar	6/6 PASS (Playwright)
API Test	5+ endpoint (Postman/Newman)
k6 p(95) Latency	285ms (threshold: < 500ms)
Docker Image Katmanlari	Multi-stage (builder + runtime)
K8s Manifest	Deployment + Service + ConfigMap
Grafana Panel	4 panel (latency, errors, RPS, business)
Bonus	OpenTelemetry Tracing (+5) + ArgoCD GitOps (+5)

7.2 Karsilasilan Zorluklar

- **JavaScript SyntaxError:** base.html ve habit_detail.html'de const token iki kez tanimlanmis, butun script blogu cokuyordu. Playwright pageerror event'i ile tespit edildi.
- **CI Pipeline Tutarsizligi:** smoke/load-test job'lari docker build yaparken build job'undan farkli imaj olusuyordu. GHCR imajini yeniden kullanan akis tasarlanarak cozuldu.
- **K8s DNS Sorunu:** configmap.yaml'da AWS_ENDPOINT_URL olarak "host.docker.internal" kullanilmisti; K8s pod'larindan bu adrese ulasilamiyor. "localstack" servis DNS adi ile duzeltildi.
- **Testcontainers Windows Encoding:** psycpg2, Windows Turkce locale nedeniyle hostname'i UTF-8 olarak ceviremiyordu. Linux/CI ortaminda calisacak sekilde platformdan bagimsiz yapi kuruldu.

7.3 Ileride Yapilabilecekler

- PgBouncer connection pooling eklenebilir — yuksek yuk altinda DB baglanti overhead azalir
- Helm chart ile K8s deploy paketi olusturulabilir (+5 bonus)
- KEDA ile queue uzunluguna gore otomatik scaling eklenebilir (+5 bonus)
- Coverage %70 kalitatif olarak yeterli ancak S3 ve OTEL kod yollari icin mock testler eklenebilir

8. Kaynaklar

Kaynak	URL
FastAPI Docs	https://fastapi.tiangolo.com
Testcontainers Python	https://testcontainers-python.readthedocs.io
k6 Documentation	https://k6.io/docs
OpenTelemetry Python	https://opentelemetry-python.readthedocs.io
Playwright Python	https://playwright.dev/python
GitHub Actions Docs	https://docs.github.com/actions

Kubernetes Docs	https://kubernetes.io/docs
Prometheus Client Python	https://github.com/prometheus/client_python